

Hand following

1. Program Instruction

What are the applications of hand following?

Virtual reality: In virtual reality games, it allows players to control game characters' movements, attacks, and other actions through gestures.

Medical field: It can be used in rehabilitation training to help patients recover hand function.

Education field: It allows students to interactively learn through gestures.

Smart homes: It allows users to control home devices' switches, adjustments, and other operations through gestures.

Industrial production: It allows workers to control robots' operations through gestures, improving production efficiency.

The hand feature detection algorithm library used in this robot is MediaPipe. MediaPipe is an open-source framework of multi-media machine learning models. Cross-platform MediaPipe can run on mobile devices, workspace and servers, as well as support mobile GPU acceleration. It is also compatible with TensorFlow and TF Lite Inference Engine, and all kinds of TensorFlow and TF Lite models can be applied on it. Besides, MediaPipe supports GPU acceleration of mobile and embedded platform.

First, a hand recognition model needs to be built, and the topic messages published by the camera node need to be subscribed to obtain the image. Then, the image is processed by flipping and other methods to detect the hand information in the image. Next, the position of the hand center point is obtained based on the connection of the hand key points. Finally, the monocular camera pan-tilt is controlled to follow the center point of the recognized hand to move up and down.

2. Operation Steps

Note:

1.The input command should be case sensitive, and keywords can be complemented using Tab key.

2.This game is only applicable to 2D pan-tilt monocular camera version.

1) Start the robot and connect it to the remote control software VNC.

2) Click on  on the system desktop to open the command line terminal.

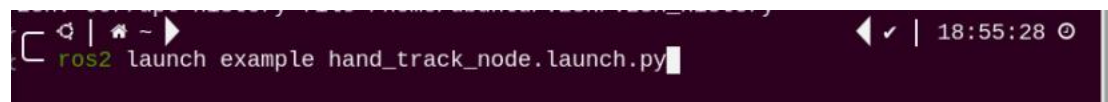
3) Run the command to stop the automatic startup service of the app.

```
~/stop_ros.sh
```

```
> ~/.stop_ros.sh
```

4) Enter the command to start the game.

```
ros2 launch example hand_track_node.launch.py
```

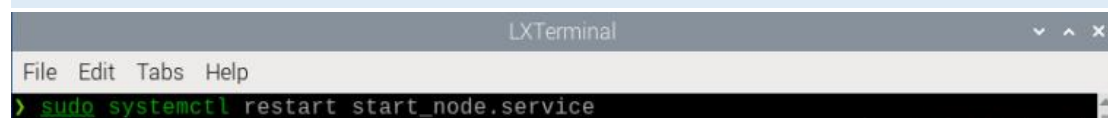


5) If you want to exit the game, press “Ctrl+C” in the terminal interface.

After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.

Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

```
sudo systemctl restart start_node.service
```



3. Program Outcome

After starting the game, the 2D pan-tilt will return to its initial position. Place your hand in front of the camera of the 2D pan-tilt, and move your hand up and down. The 2D pan-tilt will follow your hand to move up and down.

Note: Displaying the live camera feed in this game may cause the program to get stuck.

Therefore, it is not recommended to display the feedback image during the game. If you need to view the live camera feed, open a new command line terminal and enter the command “`rqt_image_view`”.

4. Program Analysis

The source code of the program is located at:

`/home/ubuntu/ros2_ws/src/example/example/hand_track/hand_track_node.py`

◆ Subscribe and publish node

A publisher message “`/ros_robot_controller/pwm_servo/set_state`” is published to control the rotation of the pan-tilt servo.

A topic message of the hand detection “`/hand_detect/center`” is subscribed to obtain the center point of the hand.

```
40 self.pwm_pub = self.create_publisher(SetPwMServoState, 'ros_robot_controller/pwm_servo/set_state', 10)
41 self.image_sub = self.create_subscription(Point2D, '/hand_detect/center', self.get_hand_callback, 1)
```

◆ Update PID data

The PID algorithm is used to update the coordinates of the X and Z axes.

```

81         self.pid_y.SetPoint = self.center.width / 2
82         self.pid_y.update(self.center.width - self.center.x)
83         self.y_dis += self.pid_y.output
84         if self.y_dis < 800:
85             self.y_dis = 800
86         if self.y_dis > 1900:
87             self.y_dis = 1900
88
89         self.pid_z.SetPoint = self.center.height / 2
90         self.pid_z.update(self.center.y)
91         self.z_dis += self.pid_z.output
92         if self.z_dis > 1900:
93             self.z_dis = 1900
94         if self.z_dis < 800:
95             self.z_dis = 800

```

The center position coordinate of the image is obtained by dividing the height or width of the image by 2. Compare the result with the center point coordinate of the hand to obtain the deviation for updating the PID data. The output result of the PID algorithm updates the coordinates of the X and Z axes.

◆ Publish pan-tilt servo angle

Call custom servo message type to publish the position information of the pan-tilt servo.

```

101         msg = SetPWMServoState()
102         msg.duration = 0.2
103         move_list = [self.z_dis, self.y_dis]
104         for i in range(2):
105             pos = PWMServoState()
106             pos.id = [i+1]
107             pos.position = [int(move_list[i])]
108             position_list.append(pos)
109
110         msg.state = position_list
111         self.pwm_pub.publish(msg)

```